

OptiCrop: Smart Agricultural Production optimization engine

Project Description:

The project Smart Agricultural Production Optimization Engine” aims to develop an advanced software system that utilizes data-driven insights to optimize agricultural production for different crops. By integrating key environmental factors such as Nitrogen (N), Phosphorous (P), Potassium (K) levels, soil temperature, humidity, pH, rainfall, and crop types, OptiCrop seeks to provide intelligent recommendations to farmers for maximizing yields and resource efficiency. The knowledge and insights gained through the OptiCrop project can be utilized by agricultural researchers, policymakers, and stakeholders to advance the understanding of crop-environment interactions and inform the development of evidence-based agricultural strategies. The OptiCrop project revolutionizes agricultural practices by providing farmers with a smart production optimization engine. By integrating key environmental factors and crop-specific recommendations, OptiCrop enables farmers to achieve optimal yields, improve resource efficiency, and contribute to sustainable agricultural practices.

Scenarios

Scenario 1: Smart Crop Recommendation for Farmers

A farmer enters soil and environmental details such as Nitrogen (N), Phosphorous (P), Potassium (K), temperature, humidity, pH level, and rainfall. The system analyzes the data and recommends the most suitable crop for maximum yield and better farming efficiency.

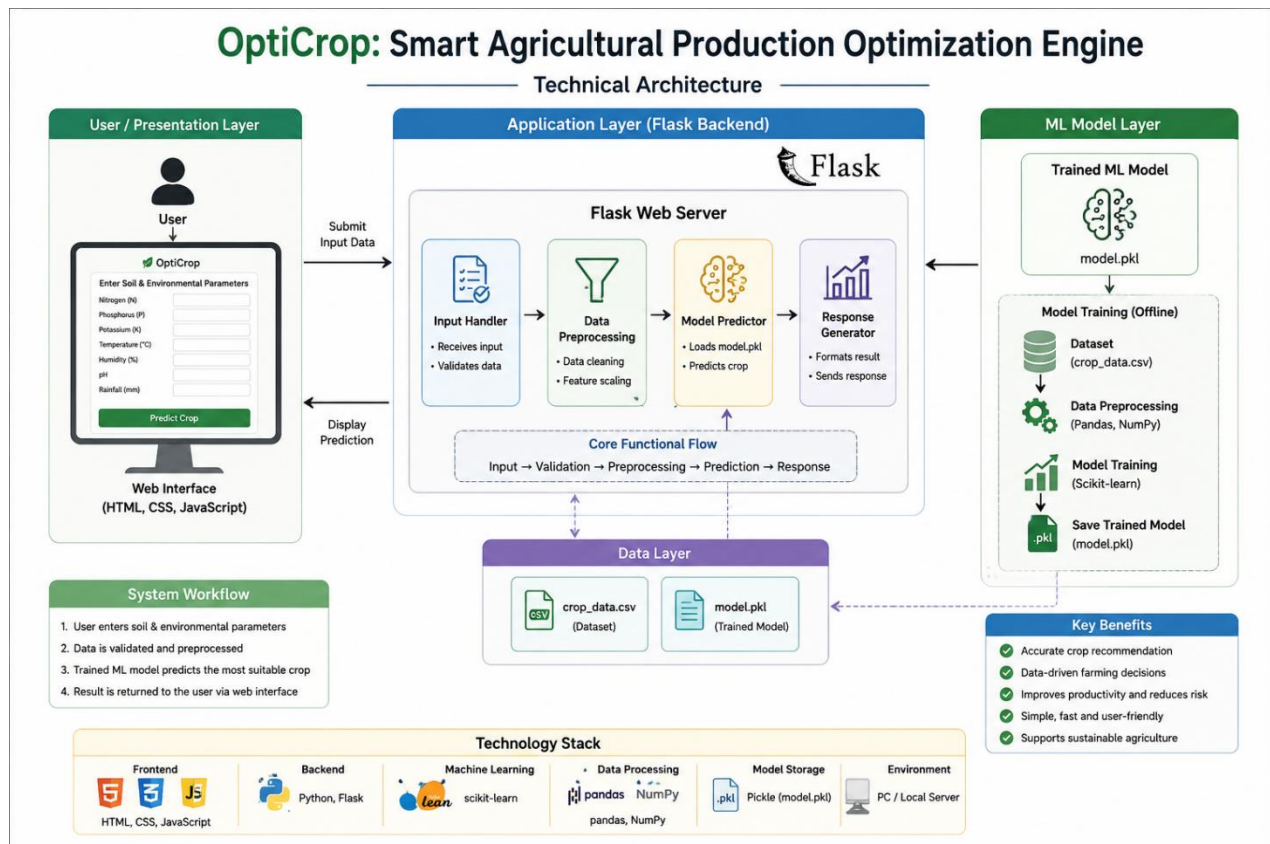
Scenario 2: Crop Suitability and Environmental Assessment

A user wants to understand whether the current soil and climate conditions are suitable for a particular crop. The application evaluates the input parameters and provides insights about crop compatibility, environmental suitability, and productivity potential.

Scenario 3: Agricultural Research and Policy Planning

An agricultural researcher or policymaker uses the system to analyze crop-environment relationships and identify patterns in agricultural production. The platform helps in making data-driven decisions for sustainable farming strategies and resource optimization.

Technical Architecture:



Pre-requisites:

- **Flask Framework Knowledge:** [Flask Documentation](#)
- **HTML, CSS, Skills:** [W3Schools](#)
- **Python Programming Proficiency:** [Python Documentation](#)
- **Version Control with Git:** [Git Documentation](#)
- **Development Environment Setup:** [Flask Installation Guide](#)

Project Workflow:

Step 1: Problem Identification

- Identify the challenge of selecting suitable crops based on soil and environmental conditions.
- Define the project objective to improve agricultural productivity using AI.

**Step 2: Requirement Analysis**

- Gather functional and non-functional requirements.
- Identify required input parameters (N, P, K, temperature, humidity, pH, rainfall).

**Step 3: Dataset Collection**

- Collect an agricultural dataset containing soil parameters and crop labels.
- Verify the quality and completeness of the data.

**Step 4: Data Preprocessing**

- Clean the dataset by removing missing or inconsistent values.
- Prepare the data for machine learning.

**Step 5: Machine Learning Model Development**

- Train the crop recommendation model using Scikit-learn.
- Evaluate different algorithms and select the best-performing model.

**Step 6: Save the Trained Model**

- Save the trained model as **model.pkl** for future predictions.

**Step 7: Frontend Development**

- Design the user interface using **HTML, CSS, and JavaScript**.
- Create forms for users to enter soil and environmental data.

**Step 8: Backend Development**

- Develop the Flask application.
- Connect the frontend with the machine learning model.

**Step 9: Model Integration**

- Load the trained model into the Flask application.
- Process user input and generate crop predictions.

**Step 10: Testing**

- Perform functional, integration, UI, and validation testing.

- Fix bugs and improve application performance.



Step 11: Documentation

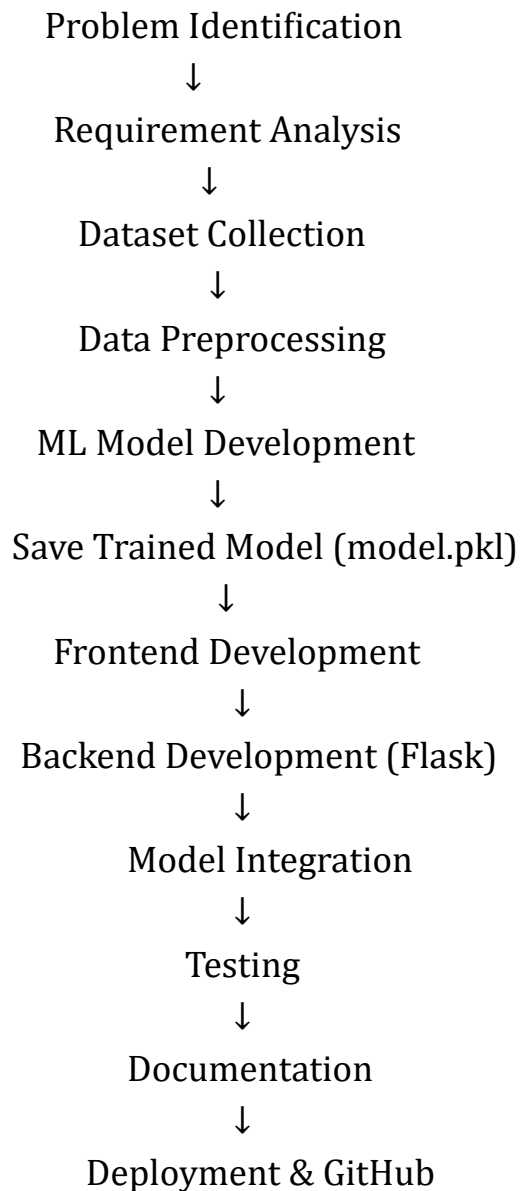
- Prepare project documentation, diagrams, testing reports, and the README file.



Step 12: Deployment

- Run the project locally and optionally deploy it to a cloud platform such as Render or PythonAnywhere.
- Publish the source code on GitHub.

Development workflow:



Technology:

Technology	Category	Purpose
Python	Programming Language	Develops the backend logic and machine learning functionality.
Flask	Web Framework	Connects the frontend with the machine learning model and handles user requests.
HTML5	Frontend	Creates the structure of the web application.
CSS3	Frontend	Designs and styles the user interface.
JavaScript	Frontend	Adds interactivity and client-side validation.
Scikit-learn	Machine Learning	Trains and predicts the most suitable crop.
Pandas	Data Processing	Reads, cleans, and processes the agricultural dataset.
NumPy	Numerical Computing	Performs mathematical and array operations for data processing.
Pickle (model.pkl)	Model Storage	Stores the trained machine learning model for prediction.
CSV Dataset	Data Source	Contains soil, environmental, and crop data used for model training.
Git	Version Control	Tracks project changes during development.
GitHub	Repository Hosting	Stores, manages, and shares the project online.
Visual Studio Code	IDE	Used to write, edit, and debug the project code.

Set Up the Environment for Starting the OptiCrop Project

Step 1: Install Python

- Download and install **Python 3.x**.
- During installation, enable "**Add Python to PATH**".

Step 2: Install Visual Studio Code

- Download and install **Visual Studio Code (VS Code)**.
- Install the **Python Extension** from the VS Code Extensions Marketplace.

Step 3: Create the Project Folder

Create a new folder for the project.

OptiCrop/

Step 4: Open the Project

Open the **OptiCrop** folder in Visual Studio Code.

File → Open Folder → OptiCrop

Step 5: Create a Virtual Environment

Open the terminal in VS Code and run:

```
python -m venv venv
```

Activate the virtual environment.

Step 6: Install Required Libraries

Install the required dependencies.

```
pip install flask pandas numpy scikit-learn
```

```
pip install -r requirements.txt
```

Step 7: Create the Project Structure:

```
OptiCrop/  
├── app.py  
├── model.pkl  
├── requirements.txt  
├── README.md  
├── dataset/  
│   └── crop_data.csv  
├── static/  
│   ├── css/  
│   ├── js/  
│   └── images/  
└── templates/  
    ├── index.html  
    └── result.html
```

Step 8: Develop the Application

- Design the frontend using **HTML, CSS**

```

index.html > html > head > style > body
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8" />
5      <meta name="viewport" content="width=device-width, initial-scale=1.0" />
6      <title>Opti Crop Predictor</title>
7      <style>
8          * {
9              margin: 0;
10             padding: 0;
11             box-sizing: border-box;
12             font-family: "Segoe UI", Tahoma, Geneva, Verdana, sans-serif;
13         }
14
15         body {
16             min-height: 100vh;
17             background:
18                 linear-gradient(
19                     rgba(0, 0, 0, 0.35),
20                     rgba(0, 0, 0, 0.35))

```

Develop the backend:

Developing with flask backend

```

app.py > ...
1  from flask import Flask, render_template, request
2  import joblib
3  import numpy as np
4
5  app = Flask(__name__)
6
7  # Load trained model
8  model = joblib.load("crop_model.pkl")
9

```

```
@app.route("/")
def home():
    return render_template("index.html")

@app.route("/predict", methods=["POST"])
def predict():
    try:
        # Get values from HTML form
        nitrogen = float(request.form["nitrogen"])
        phosphorous = float(request.form["phosphorous"])
        potassium = float(request.form["potassium"])
        temperature = float(request.form["temperature"])
        humidity = float(request.form["humidity"])
        ph = float(request.form["ph"])
        rainfall = float(request.form["rainfall"])

        # Create input array
        features = np.array([
            nitrogen,
            phosphorous,
            potassium,
            temperature,
            humidity,
            ph,
            rainfall
        ])
    ])
```

Machine learning modal :

Model Selection

To identify the most accurate crop recommendation model, multiple machine learning algorithms were trained and evaluated using the agricultural dataset. Each model was tested based on its prediction accuracy and overall performance. The

algorithm with the highest accuracy and reliable results was selected for deployment in the OptiCrop application.

Recommended Dataset

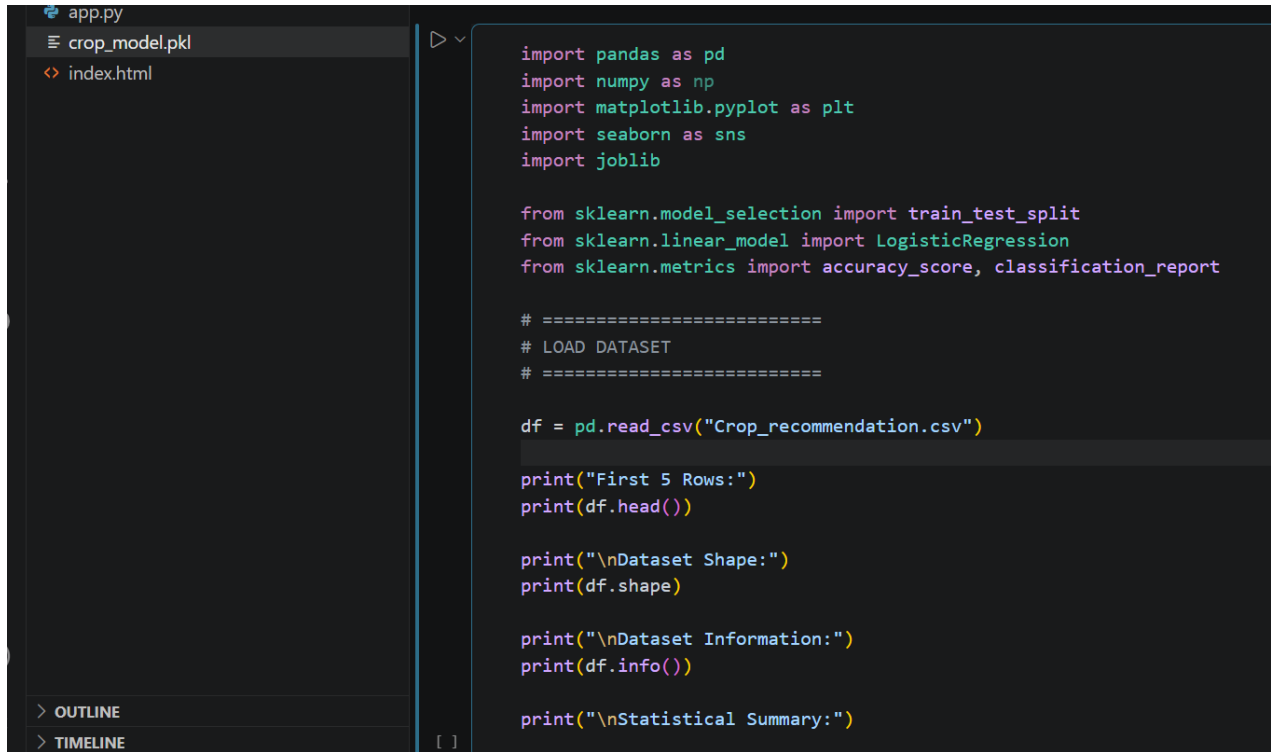
Crop Recommendation Dataset (Kaggle)

This dataset contains:

- **Nitrogen (N)**
- **Phosphorus (P)**
- **Potassium (K)**
- **Temperature**
- **Humidity**
- **pH**
- **Rainfall**
- **Crop Label (Target)**

It is widely used for machine learning-based crop recommendation projects and is based on agricultural, rainfall, climate, and fertilizer data.

Train the model:



```
app.py
└─ crop_model.pkl
   └─ index.html

> OUTLINE
> TIMELINE

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import joblib

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report

# =====
# LOAD DATASET
# =====

df = pd.read_csv("Crop_recommendation.csv")

print("First 5 Rows:")
print(df.head())

print("\nDataset Shape:")
print(df.shape)

print("\nDataset Information:")
print(df.info())

print("\nStatistical Summary:")
```

```

crop_model.pkl
index.html

# =====
# CHECK NULL VALUES
# =====

print("\nNull Values:")
print(df.isnull().sum())

# =====
# REMOVE DUPLICATES
# =====

print("\nDuplicate Rows:", df.duplicated().sum())

df = df.drop_duplicates()

print("Shape After Removing Duplicates:", df.shape)

# =====
# UNIVARIATE ANALYSIS
# =====

plt.figure(figsize=(12, 5))
sns.countplot(x='label', data=df)
plt.xticks(rotation=90)
plt.title("Crop Distribution")
plt.show()

```

> OUTLINE

```

index.html

# TRAIN TEST SPLIT
# =====

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

print("\nTraining Data:", X_train.shape)
print("Testing Data:", X_test.shape)

# =====
# LOGISTIC REGRESSION MODEL
# =====

model = LogisticRegression(max_iter=5000)

print("\nTraining Model...")
model.fit(X_train, y_train)

# =====
# PREDICTION
# =====

```

> OUTLINE
> TIMELINE

crop_model.pkl
 index.html

```

print("\nModel Accuracy:")
print(round(accuracy * 100, 2), "%")

print("\nClassification Report:")
print(classification_report(y_test, predictions))

# =====
# SAVE MODEL
# =====

joblib.dump(model, "crop_model.pkl")

print("\nModel Saved Successfully!")
print("File Name: crop_model.pkl")

```

[]

...

First 5 Rows:

	N	P	K	temperature	humidity	ph	rainfall	label
0	90	42	43	20.879744	82.002744	6.502985	202.935536	rice
1	85	58	41	21.770462	80.319644	7.038096	226.655537	rice
2	60	55	44	23.004459	82.320763	7.840207	263.964248	rice
3	74	35	40	26.491096	80.158363	6.980401	242.864034	rice
4	78	42	42	20.130175	81.604873	7.628473	262.717340	rice

OUTLINE

Running application in local:

Steps After Training the Model

1. Evaluate the Model

- Test the model using unseen data.
- Measure accuracy and other performance metrics.
- Select the best-performing model.

2. Save the Trained Model

- Save the model as model.pkl using Pickle.
- This allows the model to be reused without retraining.

3. Develop the Backend

- Create the Flask application (app.py).
- Load the saved model.
- Handle user requests and predictions.

4. Develop the Frontend

- Design the user interface using HTML, CSS, and JavaScript.
- Create forms for entering soil and environmental data.

5. Integrate the Model

- Connect the frontend with the Flask backend.
- Pass user inputs to the trained model.
- Receive the predicted crop from the model.

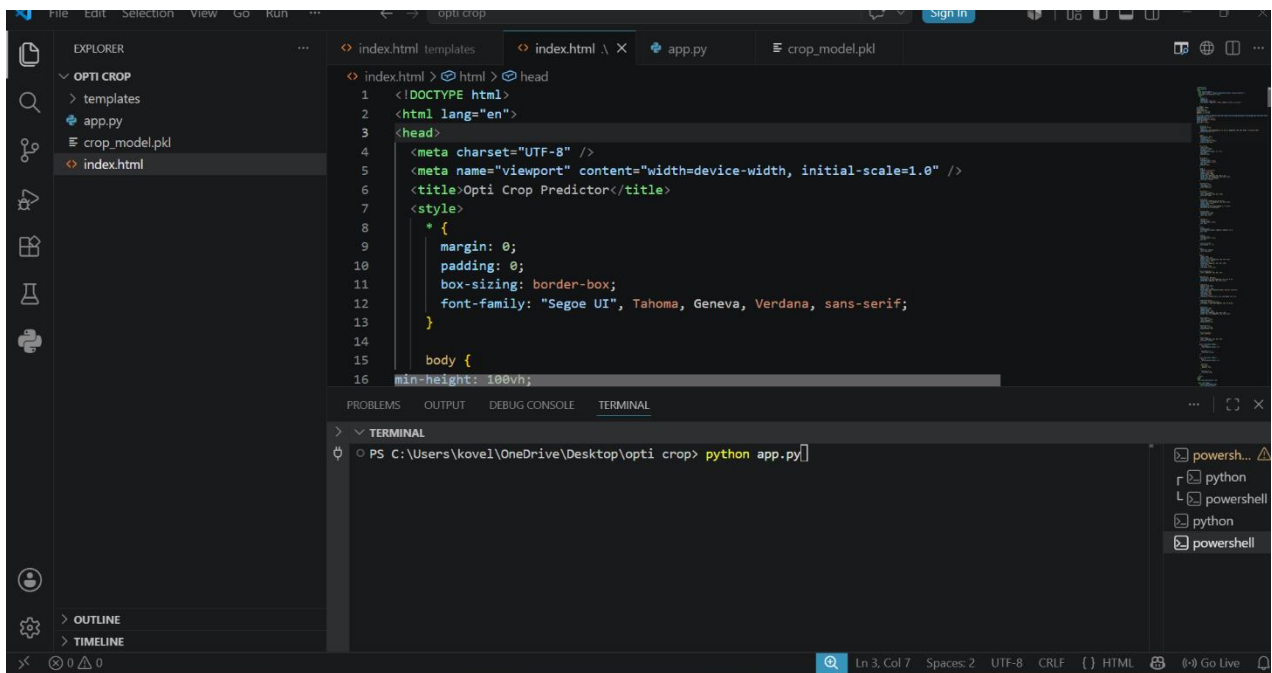
6. Test the Application

- Verify that all features work correctly.
- Check prediction accuracy, input validation, and frontend-backend communication.

7. Deploy the Project

- Run the application locally or deploy it to a hosting platform.
- Upload the source code to GitHub for version control and sharing.

Running application in local:



The screenshot shows a Visual Studio Code editor with the following components:

- EXPLORER:** A file tree on the left showing the project structure: `OPTI CROP` (parent), `templates` (subdirectory), `app.py` (Python file), `crop_model.pkl` (pickle file), and `index.html` (HTML file).
- EDITOR:** The `index.html` file is open, showing HTML code with a `<head>` section containing meta tags for charset, viewport, and title, and a `<style>` block with CSS rules for margin, padding, box-sizing, and font-family. The `<body>` section is partially visible.
- TERMINAL:** A terminal window at the bottom shows the command `python app.py` being executed in a PowerShell prompt.

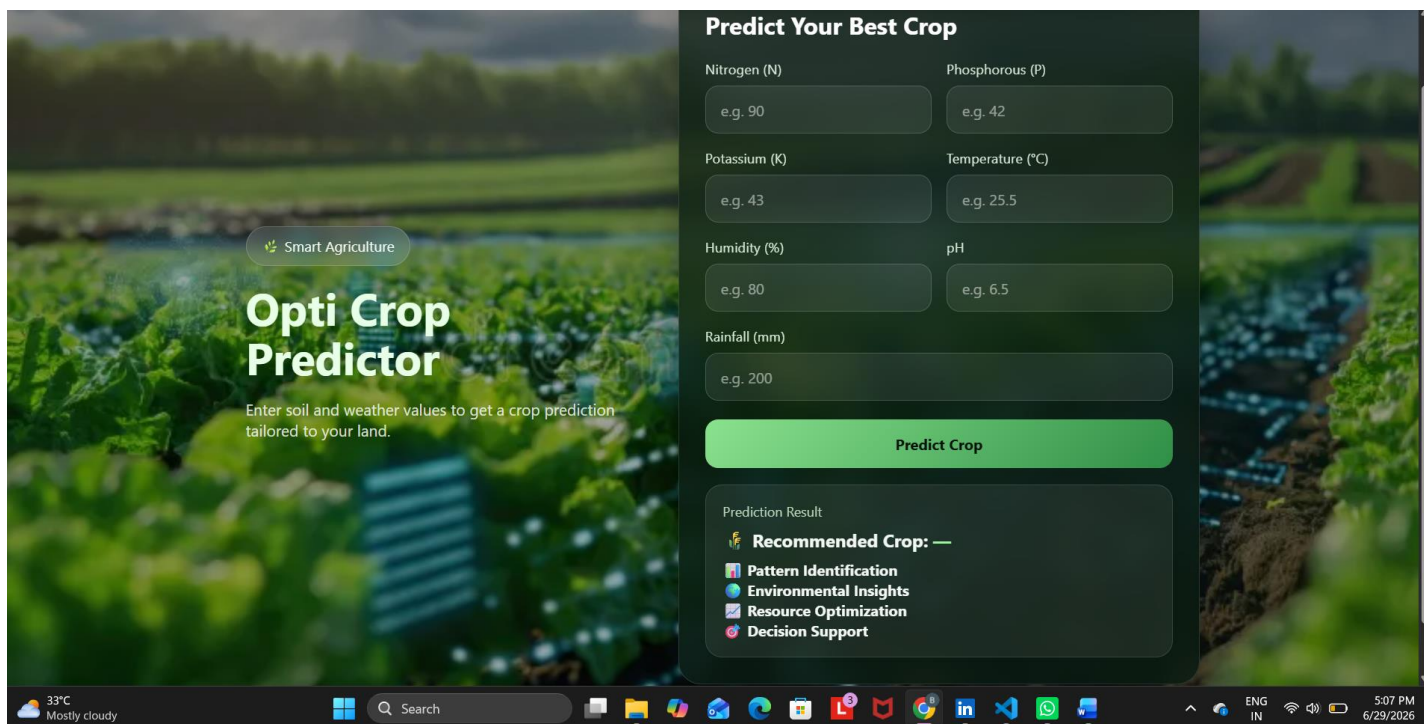
```
PS C:\Users\kovel\OneDrive\Desktop\opti_crop> python app.py
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 989-351-582
```

Project testing:

Run the application in the localhost



Frontend:



Open Project



Activate Virtual Environment



Install Dependencies



Run app.py



Open <http://127.0.0.1:5000>



Enter Input Values

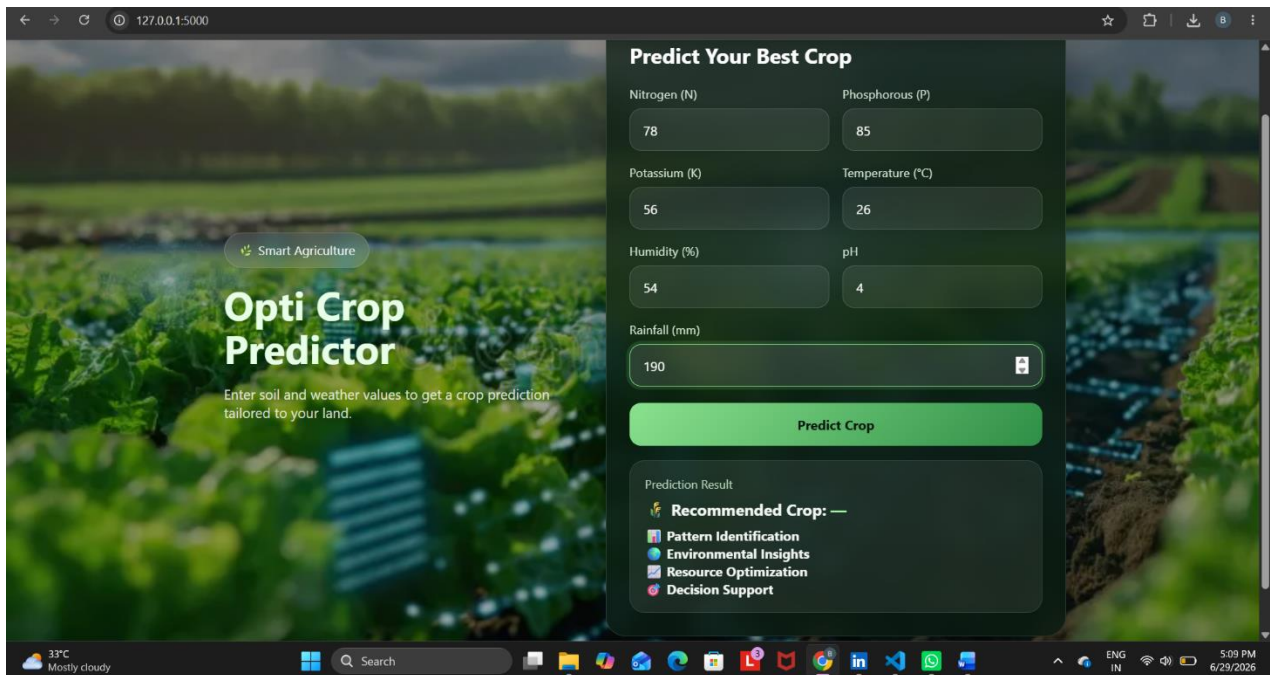


Click "Predict"



View Recommended Crop

- Enter the required soil and environmental parameters:
 - Nitrogen (N)
 - Phosphorus (P)
 - Potassium (K)
 - Temperature
 - Humidity
 - pH
 - Rainfall
- Click the **Predict** button.



The screenshot shows a web browser window displaying the "Opti Crop Predictor" application. The interface is split into two main sections. On the left, a large image of a green field with a blue irrigation system is overlaid with the text "Smart Agriculture" and "Opti Crop Predictor". Below this, it says "Enter soil and weather values to get a crop prediction tailored to your land." On the right, a dark-themed panel titled "Predict Your Best Crop" contains input fields for various parameters. The values entered are: Nitrogen (N) 78, Phosphorous (P) 85, Potassium (K) 56, Temperature (°C) 26, Humidity (%) 54, pH 4, and Rainfall (mm) 190. A green "Predict Crop" button is located below these inputs. Under the button, a "Prediction Result" section shows a "Recommended Crop: —" and a list of four categories: Pattern Identification, Environmental Insights, Resource Optimization, and Decision Support. The browser's address bar shows "127.0.0.1:5000". The Windows taskbar at the bottom shows the date and time as 5:09 PM on 6/29/2026, along with various system icons and a search bar.

127.0.0.1:5000

Smart Agriculture

Opti Crop Predictor

Enter soil and weather values to get a crop prediction tailored to your land.

Predict Your Best Crop

Nitrogen (N)	Phosphorous (P)
78	85
Potassium (K)	Temperature (°C)
56	26
Humidity (%)	pH
54	4
Rainfall (mm)	
190	

Predict Crop

Prediction Result

Recommended Crop: —

- Pattern Identification
- Environmental Insights
- Resource Optimization
- Decision Support

33°C Mostly cloudy

Search

ENG IN 5:09 PM 6/29/2026

Crop predicted:



Results:

- Displays the predicted crop based on the entered values.
- Generates accurate recommendations using the trained machine learning model.
- Shows the result instantly on the web page.
- Helps users make informed agricultural decisions.